Accelerating Particle-Mesh Gravity Simulations with OpenMP and MPI

Lev Kruglyak

May 9, 2025

1 Introduction

Large-scale gravitational simulations have become an indispensable tool of precision cosmology, connecting our understanding of the early universe with the large scale structures we see today. In the standard model of cosmology, the universe begins almost perfectly uniformly, with tiny density perturbations seeded by inflationary expansion. Under the influence of gravity and with the expansion of the universe, these initial inhomogeneities are magnified into a web of galaxy clusters and voids. Simulating such evolution is typically very expensive to run, both from a compute and memory perspective, and so this is an excellent candidate for massive parallelization.

Two-dimensional toy models can provide valuable insight into algorithmic behavior, scaling properties, and qualitative features of clustering with far lower computational cost and simpler implementation details. In this project we implement a highly parallel large-scale gravitational solver, using the particle-mesh (PM) algorithm to simulate the evolution of a flat, two-dimensional, expanding, dark-matter dominated universe with periodic boundary conditions.

1.1 The Particle-Mesh Algorithm

In a classical N-body simulation we simulate N particles each with mass m_i and position $\boldsymbol{x}_i$. The particles obey Newton's law of gravitation, i.e.

$$m_i \frac{\partial^2 \boldsymbol{x}_i}{\partial t^2} = -G\, m_i \sum_{j \neq i} m_j \frac{\boldsymbol{x}_i - \boldsymbol{x}_j}{|\boldsymbol{x}_i - \boldsymbol{x}_j|^3},$$

so the force evaluation scales as $\mathcal{O}(N^2)$. For cosmological problems where N can exceed 10^8, the quadratic cost and memory traffic of computing all pairwise interactions becomes computationally unfeasible.

A useful alternative is to regard the system as a collisionless fluid characterized by its mass density field $\rho(\boldsymbol{x})$. By the Poisson equations, the gravitational potential $\phi(\boldsymbol{x})$ satisfies

$$\nabla^2 \phi(\boldsymbol{x}) = 4\pi G \rho(\boldsymbol{x}).$$

and the force at each point is then $\boldsymbol{F}(\boldsymbol{x}) = -\nabla\phi(\boldsymbol{x})$. If we work in a flat, periodic square of side length L, every function can be represented by its complex Fourier transform

$$f(\boldsymbol{x}) = \int \hat{f}(\boldsymbol{k})\, e^{i\boldsymbol{k}\cdot\boldsymbol{k}}\, \mathrm{d}^2\boldsymbol{k}, \qquad \hat{f}(\boldsymbol{k}) = L^{-2} \int f(\boldsymbol{x})\, e^{-i\boldsymbol{k}\cdot\boldsymbol{x}}\, \mathrm{d}^2\boldsymbol{x},$$

with wave-vectors $\boldsymbol{k} = 2\pi L^{-1}(n_x, n_y)$ for $n_x, n_y \in \mathbb{Z}$. Under this transform, the Laplacian operator ∇^2 becomes multiplication by $-|\boldsymbol{k}|^2$, and so for every non-zero Fourier node we have

$$\hat{\phi}(\boldsymbol{k}) = -\frac{4\pi G}{|\boldsymbol{k}|^2}\, \hat{\rho}(\boldsymbol{k}), \qquad \boldsymbol{k} \neq 0,$$

while the $\boldsymbol{k} = \boldsymbol{0}$ (mean-density) mode is set to zero to avoid a singularity. The gradient operator ∇ in Fourier space becomes multiplication by $i\boldsymbol{k}$ (we get two components here). Thus, once the potential is known spectrally, its gradient can be computed in Fourier space and mapped back to ordinary space as

$$\begin{aligned}
\widehat{\boldsymbol{F}}(\boldsymbol{k}) &= \mathrm{i}\boldsymbol{k}\hat{\phi}(\boldsymbol{k}) = -\frac{4\pi i G \boldsymbol{k}}{|\boldsymbol{k}|^2}\, \hat{\rho}(\boldsymbol{k}) \\
\boldsymbol{F}(\boldsymbol{x}) &= \int i\boldsymbol{k}\widehat{\phi}(\boldsymbol{k}) e^{i\boldsymbol{k}\cdot\boldsymbol{x}} \mathrm{d}^2\boldsymbol{k} = -\int \frac{4\pi i G \boldsymbol{k}}{|\boldsymbol{k}|^2} e^{i\boldsymbol{k}\cdot\boldsymbol{x}}\, \mathrm{d}^2\boldsymbol{k}.
\end{aligned}$$

In other words, both "inverting" the Laplacian and computing the potential gradient boil down to simple, element-wise multiplications in Fourier space. These analytic solutions to the Poisson equation can be numerically solved by adding "tracer" particles which sample the density field. This coupling of particle with field is known as the particle-mesh algorithm.

The PM algorithm starts by populating the domain with N tracer particles that sample the prescribed surface-density field. We then enter a timestep loop by which we we evolve the universe. During each step the particle masses are deposited onto a uniform mesh with a Cloud-In-Cell (CIC) kernel, yielding the mesh density values $\rho(\boldsymbol{x})$. We then Fourier-transform this 2d texture, apply the mode-by-mode Laplacian inversion and spectral gradient described above to obtain $\widehat{\boldsymbol{F}}(\boldsymbol{k})$, and inverse-transform to recover the force field $\boldsymbol{F}(\boldsymbol{x})$ on the mesh. Finally, CIC interpolation maps the mesh forces back to the individual particles. Particle positions and velocities are advanced with a second-order symplectic leapfrog in the using kick–drift–kick sequence: a half-step velocity kick using the old force, a full position drift, recomputation of forces via the mesh, and a final half-kick. Finally, periodic boundary conditions wrap any coordinate that exits the box $[0, L)$.

2 Implementation

The solver is written in modern `C++20` and built with `CMake`. It uses a hybrid **MPI + OpenMP** model: MPI distributes independent slabs of the mesh across nodes, while OpenMP exploits the many cores available within each node. The only external dependencies are `FFTW` (MPI and OpenMP flavours), `OpenMP`, `MPI`, `glm` for algebra, and `nlohmann/json` for run-time configuration; all are fetched at configure time via `FetchContent`. The rank 0 node is reserved for I/O and visualisation, with the remaining nodes used for computation.

2.1 Domain Decomposition

A one–dimensional slab decomposition is adopted. For a mesh of $N_x \times N_y$ cells every worker rank owns a contiguous strip of lN_x columns starting at global index lx_0. This choice keeps the real-space arrays contiguous in the y direction and avoids the expensive all-to-all communication required by a full 2-D pencil layout. Two guard columns are stored on either side of each slab so that particle deposition and force interpolation never read or write remote memory. Only the guard columns are exchanged with the left and right neighbours at each time-step using `MPI_Sendrecv`.

2.2 Particle Generation and Sampling

At initialization the code must generate $N_p \times N_x \times N_y$ tracer particles that sample an analytically defined surface–density field $\rho_{\text{init}}(x, y)$, where N_p is the number of particles per pixel. We tile the unit square $[0, L) \times [0, L)$ with a regular grid of candidate positions and then apply an acceptance–rejection scheme: for each grid point (x_i, y_j), we draw a uniform random number $u \in [0, \rho_{\max}]$ (with $\rho_{\max}$ given by the known maximum of ρ_{init}) and accept the sample if $u < \rho_{\text{init}}(x_i, y_j)$. Accepted points are stored in the host vector `particles` until the appropriate number of particles is generated. This approach ensures that the empirical distribution of particles exactly matches the target density, while avoiding the complexity of more advanced sampling methods.

The random streams on each MPI rank are seeded from a deterministic combination of a global seed, the MPI rank, and the OpenMP thread ID. This guarantees reproducible, independent samples across ranks and threads. Before entering the time-stepping loop, each rank contributes its local batch of particles to a global count via `MPI_Allreduce`, enabling diagnostics on the achieved sampling accuracy and total particle number.

2.3 Mass Deposition and Halo Exchange

Particle–to–mesh (CIC) deposition is perfectly parallelizable, so an `omp parallel for` loop walks the local particle array and updates the four nearest mesh nodes with a thread-private accumulator. Within a rank all threads write directly into the shared buffer, so no threading synchronisation is needed. Mass that spills into a guard column is collected in two small

send buffers, and these these are exchanged with the neighbouring ranks and added to the remote guard columns before the guard-to-core copy is performed. The resulting cost of halo exchange is $O(2N_y)$ doubles per rank, completely independent of the number of particles.

2.4 Spectral Solver

The Poisson solve is performed with the `FFTW` MPI interface:

1. Each rank copies its real ρ slab into a complex work array, and performs an in-place `c2c` Fourier transform using `FFTW`.

2. A single parallel OpenMP loop multiplies every spectral node by $-4\pi G/(k_x^2 + k_y^2 + \epsilon^2)$, computes the scaled gradients $ik_x\hat{\phi}$ and $ik_y\hat{\phi}$, and stores them in a vector field. Adding the softening term ϵ^2 prevents division by 0 and damps small-scale noise.

3. Two inverse FFTs populate the real-space force vector field.

Because FFTW's MPI back-end redistributes data internally, no explicit *transpose* is needed. The slab layout is preserved from input to output, and the only inter-rank traffic is the automatically managed all-to-all inside FFTW.

2.5 Force Interpolation and Particle Update

After the spectral solve, a second halo exchange sends one column of $\mathbf{F}$ on either side so that the CIC interpolation routine can read four force values for every particle regardless of where it sits in the cell. OpenMP is again used to update particles:

1. In the kick-drift-kick stage, the two velocity kicks and the position drift are combined in one parallel OpenMP pass over the particle array.

2. A lightweight integer flag records whether a particle crossed the left or right domain boundary. The actual migration is a two-step `MPI_Sendrecv`.

3 Results

To heuristically verify that our solver reproduces the expected cosmic-web, we initialized the particle distribution using a two-dimensional Perlin-noise field with a prescribed spectral slope. This generates smooth, multi-scale density fluctuations that mimic the scale-free nature of primordial perturbations. Figure1 shows the time evolution of the surface density: the initially correlated noise (left panels) is gradually amplified by gravity into a network of filaments and nodes separated by underdense voids (right panels). The emergent web-like patterns – filament thickness, void sizes, and connectivity – are in qualitative agreement with

those seen in full N-body cosmological simulations, giving confidence that our PM solver captures the essential physics of structure formation.

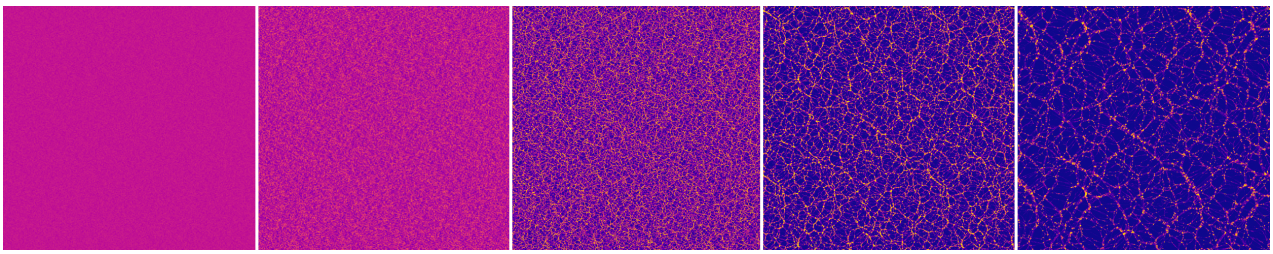

Figure 1: Structure formation as simulated by our solver.

4 Performance and Scaling Analysis

All timings represent the wall–clock seconds required for one complete PM timestep (mass assignment, FFT–based Poisson solve, force interpolation, and leapfrog update).

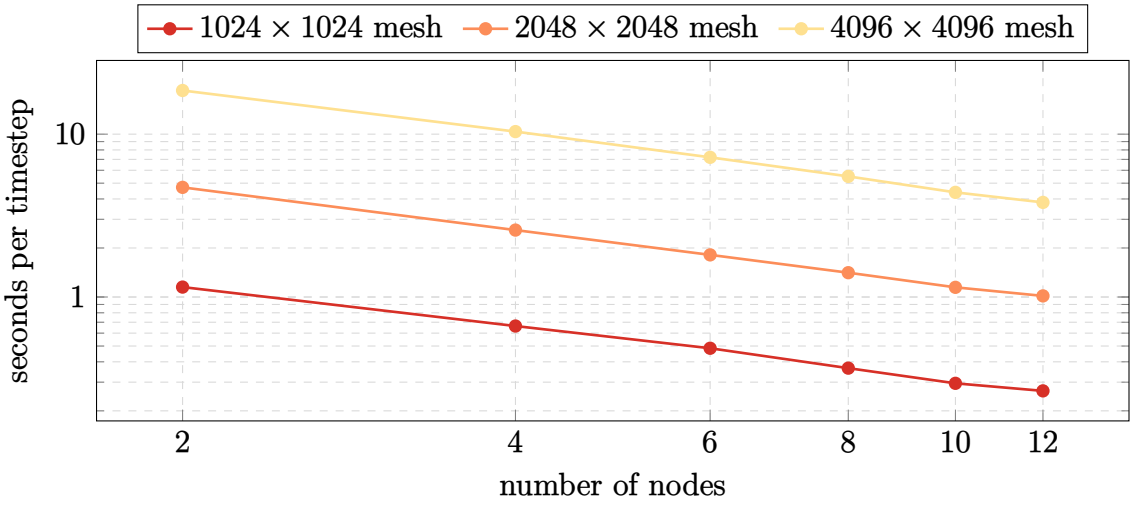

Figure 2: Timestep Duration vs Number of Nodes

Figure 2 fixes the mesh size and varies the number of compute nodes. For every resolution the timestep cost falls roughly as $\mathrm{T} \propto N_{\mathrm{node}}^{-1}$ until about ten nodes, after which the curve flattens slightly as inter-node communication begins to dominate, especially for small mesh sizes, where there is less work to be done.

Figure 3 inverts the perspective: each series holds the node count constant while increasing the mesh resolution. The plot shows a time complexity consistent with the theoretical one. Figure 4 normalises every timing series to its slowest run, highlighting the parallel efficiency. For the 4096^2 mesh we obtain nearly ideal speed-up through eight nodes and retain consistent

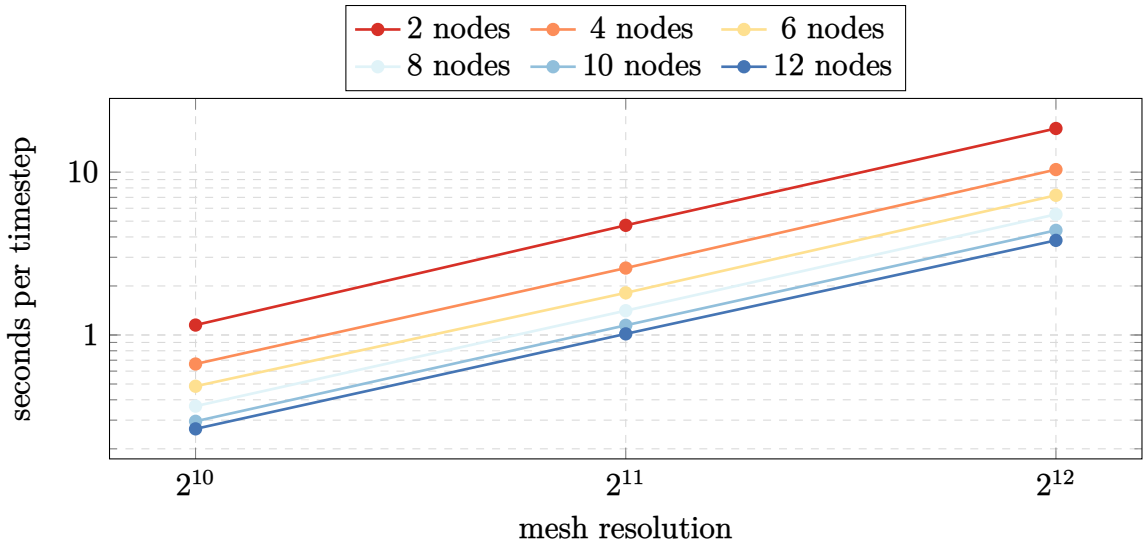

Figure 3: Timestep Duration vs Mesh Resolution

acceleration at twelve nodes. The smaller grids begin to saturate earlier, reflecting the growing share of communication versus local compute.

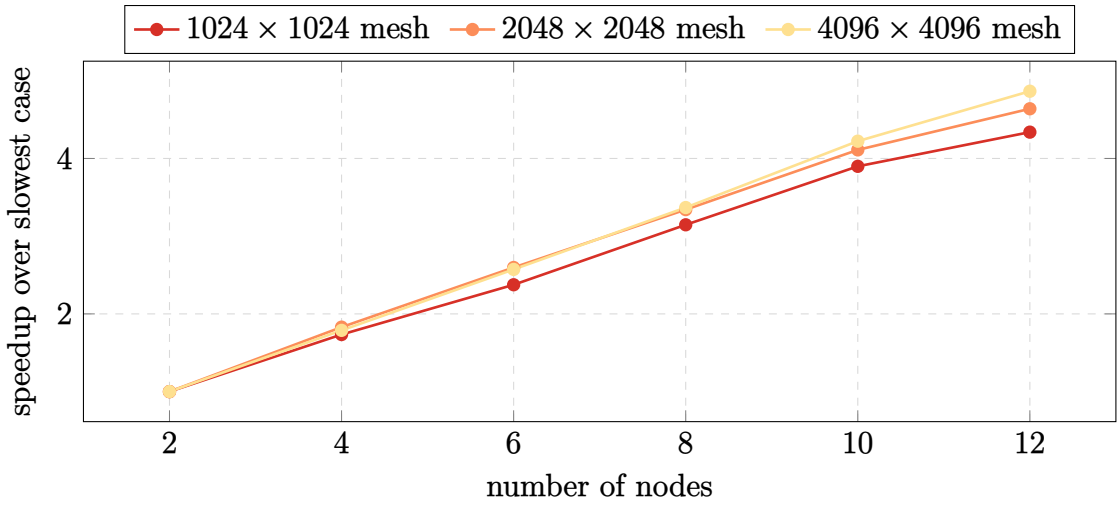

Figure 4: Speedup vs Number of Nodes

4.1 Simulation Details

Aside from varying the resolution of the mesh, the other simulation parameters were left constant. We use a 1.0×1.0 box, with total mass 1.0 and gravitational force 1.0. We add 50 particles per cell. The initial conditions use 2d Perlin noise with 4 octaves and a scale of 1.0. This contributes 0.25 perturbation to an initially uniform density.

5 Resource Justification

We would like to run a 16384×16384 mesh with 50 tracer particles per cell. We select a production-run configuration of $N = 128$ compute nodes, which will complete one full simulation (100 timesteps). This configuration simulates about 13 billion particles, and each node has enough RAM to hold around 100 million particles (< 10 Gb). Based on linear regressions ran on the benchmark data, this will take

$$0.63693(1.1037 \times 10^{-6} \cdot (16384)^2 + 0.03167)^{0.7717} \cdot \frac{1000}{3600} = 3.05 \text{ hours.}$$

As with the benchmark, we request 8 CPUs per task, and 2 tasks per node. In total, this also uses 390 node hours.